DAS 839 – NoSQL Systems

End-Semester Project — Phase 2 Final Report

Multi-Pipeline ETL and Reporting Framework for Web Server Log Analytics

Ashok (IMT2023083)

Sathvik (IMT2023001)

Sasank (IMT2023120)

Kapil (IMT2023052)

May 2026

GitHub repository: https://github.com/ashokCh-dev/No_Sql_final_project

Youtube link: https://youtu.be/ejfpx7Dp34I

Dataset: NASA HTTP Server Log (July + August 1995) — 3,457,877 records, ∼390 MB

Contents

1. Architecture

1.1 Project structure

```
final_proj/
+-- main.py                              # CLI entry point (4 pipelines
    + reporter)
+-- config.py                            # Central constants (Hadoop,
    Mongo, PG, Hive, Pig)
+-- setup.sql                            # PostgreSQL schema (run once)
+-- data/                                # NASA_access_log_{Jul95,Aug95}
+-- db/connection.py                     # psycopg2 conn +
    fetch_run_meta helper
+-- pipelines/
|    +-- common/parser.py                # Shared regex parser + HDFS
    stager
|    +-- mapreduce/  runner.py loader.py
    mr_jobs/{q1,q2,q3}_{mapper,reducer}.py
|    +-- mongodb/    runner.py loader.py
|    +-- pig/        runner.py loader.py scripts/{q1,q2,q3}.pig
|    \-- hive/       runner.py loader.py
    scripts/{q1,q2,q3,all_queries}.hql
+-- reporting/reporter.py                # Reads PostgreSQL, prints
    formatted report
\-- tools/                               # Schema migrations
```

1.2 Infrastructure

Component	Details
Hadoop	3.3.6, single-node pseudo-distributed; HDFS on `localhost:9000`
YARN	Default scheduler, single NodeManager
Pig	0.17.0, run in `-x mapreduce` mode against YARN
Hive	3.1.3, embedded Derby metastore, runs on JDK 8 (Hadoop runs on JDK 21)
MongoDB	8.0, `localhost:27017`
PostgreSQL	16.13, custom cluster at `/home/ashok_ubun/pgdata_nasa/` (port 5433)
Python	3.12 + `psycopg2-binary`, `pymongo`
Dataset	NASA HTTP logs Jul95 (1.89 M) + Aug95 (1.57 M) = 3.46 M lines

2. Parsing Approach

A single regular expression in `pipelines/common/parser.py` extracts **8 structured fields** per log line:

`host`, `log_date` (YYYY-MM-DD), `log_hour`, `http_method`, `resource_path`, `protocol_version`, `status_c`

Cleaning rules: (i) missing bytes (`-`) become 0; (ii) lines that fail the regex are *not silently dropped* — they return `None` and are counted in `malformed_count`. The same `parse_line()` function is imported by all four pipelines, so all engines see identical records.

Where parsing runs: the parser is invoked at the load stage, before records are handed to the chosen engine (HDFS TSV for MR/Pig/Hive, BSON documents for MongoDB). The same regex would be applied inside the engine (Pig's `REGEX_EXTRACT_ALL`, Hive's `RegexSerDe`, an MR mapper); we centralise it because the project statement requires the four pipelines to share identical parsing semantics, and a single Python implementation is the cleanest guarantee of

that. Result is verified by the fact that all four pipelines produce byte-identical aggregates and identical malformed counts (3,736 over Jul+Aug).

3. ETL Workflow

Every pipeline runner follows the same six-step skeleton:

1. **Preflight** — check input files exist, target service (HDFS/Mongo/Hive) is reachable.
2. **Create run record** — `INSERT INTO pipeline_runs` with `pipeline_name`, `input_file`, `started_at`; returns a new `run_id`.
3. **Batch staging** — for each input file, stream-parse lines and write them to the engine's input store (HDFS TSV for MR/Pig/Hive; MongoDB documents stamped with `run_id`+`batch_id` for Mongo). Exactly *one* row is appended to `batch_log` per input file. Records are flushed in 50 k-line chunks for memory safety (§4).
4. **Run queries** — invoke the engine's native query mechanism: *Hadoop Streaming* mapper/reducer pairs (MR), *aggregation pipeline* (Mongo), *Pig Latin script* (Pig), *HiveQL script* (Hive). The `--query` flag gates which Q1/Q2/Q3 actually runs.
5. **Load results** — pipeline-specific `loader.py` reads the engine's output (HDFS `part-*` files, Mongo cursor, etc.), parses each row, and `INSERT`s into PostgreSQL result tables. Each result row is stamped with `pipeline_name` and `execution_time` for full lineage.
6. **Finalize** — `UPDATE pipeline_runs` with `finished_at`, `runtime_seconds`, `num_batches`, `total_records`, `malformed_count`, and `avg_batch_size`.

This shared skeleton means swapping engines does not change *any* metadata, logging, or reporting code.

4. Batching Logic

Batch = one input log file. Passing `--inputs Jul95 Aug95` yields `batch_id=1` (July) and `batch_id=2` (August). This is the canonical interpretation given in the evaluation guidelines: "the two log files may be treated as two batches: Batch 1: July dataset, Batch 2: August dataset."

Why file-as-batch over fixed-size chunking? Choosing `batch_size=50000` (or any other number) over a single file is arbitrary — there is no natural answer to "why 50k vs 60k?". File-as-batch aligns batches with *meaningful data boundaries* (one calendar month per batch), is sanctioned by the eval guidelines, and remains comparable across all four pipelines because they all consume the same `--inputs` list.

Why this is also easier on the engines. A batching unit that matches a complete logical dataset (one month of records) keeps each engine's downstream machinery simple:

- **Reducers / aggregators process one complete dataset at a time.** Because all of July's records (and only July's) belong to batch 1, the MapReduce shuffle, Pig's `GROUP BY`, and Hive's `GROUP BY` each see one coherent input partition per batch. No reducer key has to be reconciled across many tiny fixed-size batches; aggregation is a single pass over a single batch's full key space.
- **No cross-batch merge step in the loader.** The result tables store one aggregate row per `(log_date, ...)` key. Because each batch already contains every record for the dates it covers, no addition/merge needs to happen *between* batch results. With fixed-size

50 k-record chunks, the same calendar day would have been split across ∼10 batches and the loader (or the engine) would have to fold them back together.

- **Fewer commit / metadata round-trips during staging.** Each batch produces exactly one `INSERT` into `batch_log` and one `COMMIT`. Two files → two metadata writes, instead of the ∼70 we used to do with 50 k-record chunking over Jul+Aug.
- **Smaller bookkeeping surface in the reporter.** `batch_log` now has one row per file, so the report's "per-batch sizes" section is short and human-readable instead of a scroll of dozens of identically-sized rows.

Implementation detail (memory bounding). Multi-million-line files cannot be held in Python memory all at once, so each runner streams records in 50 k-record buffers (`STREAM_CHUNK_SIZE` in `config.py`) before flushing to HDFS / `insert_many` to Mongo. This buffer size has no batching semantics: it is an internal flush threshold to keep RSS bounded (∼10 MB per buffer), is not user-configurable, and never produces extra rows in `batch_log`. From the engines' perspective each batch is still one logical, monolithic input partition.

Schema columns (§5): `batch_log.records_in_batch` holds the per-file record count; `pipeline_runs.num_bat` is the number of input files; `pipeline_runs.avg_batch_size` is $\frac{\texttt{total\_records}}{\texttt{num\_batches}}$.

5. Result Schema

All pipelines write to the same five PostgreSQL tables:

Table	Purpose
`pipeline_runs`	One row per run. Columns: `run_id`, `pipeline_name`, `input_file`, `started_at`, `finished_at`, `runtime_seconds`, `num_batches`, `total_records`, `malformed_count`, `avg_batch_size`.
`batch_log`	One row per input file (= per batch). Columns: `run_id`, `batch_id`, `records_in_batch`, `malformed_in_batch`.
`q1_daily_traffic`	`(run_id, pipeline_name, execution_time, log_date, status_code, request_count, total_bytes)`
`q2_top_resources`	`(run_id, pipeline_name, execution_time, rank, resource_path, request_count, total_bytes, distinct_host_count)`
`q3_hourly_errors`	`(run_id, pipeline_name, execution_time, log_date, log_hour, error_request_count, total_request_count, error_rate, distinct_error_hosts)`

Table 1: PostgreSQL schema. Result rows carry `pipeline_name` and `execution_time` directly to satisfy the project statement's requirement that "stored results must include the pipeline name, run identifier, batch identifier, and time of execution."

We chose *separate* result tables per query (not one with a `query_name` discriminator) because the queries have different column sets and per-table typing keeps queries simple. The reporter joins each result table to `pipeline_runs` by `run_id` so the report is pipeline-agnostic.

6. Query Implementation

The three required queries are defined by their output column schemas; each pipeline implements them in its own DSL.

6.1 Q1 — Daily Traffic Summary

For each `(log_date, status_code)`, compute `request_count` and `total_bytes`.

- **MR**: mapper emits `(date\tstatus, 1\tbytes)`; reducer sums.
- **Mongo**: `$match -> $group` on `(log_date, status_code)` with `$sum`.
- **Pig**: `GROUP records BY (log_date, status_code); FOREACH ... COUNT($0), SUM(bytes)`.
- **Hive**: `SELECT log_date, status_code, COUNT(*), SUM(bytes_transferred) FROM staged_logs GROUP BY ....`

6.2 Q2 — Top 20 Requested Resources

Top 20 by `request_count` per resource path, with distinct host count.

- **MR**: mapper emits `(path, host\t1\tbytes)`; reducer uses a `set` per path; loader sorts `DESC` and takes top 20.
- **Mongo**: `$group`+`$addToSet(host)` then `$sort DESC` + `$limit 20`.
- **Pig**: `GROUP BY path`; `DISTINCT records.host` inside `FOREACH`; `ORDER DESC`; `LIMIT 20`.
- **Hive**: `COUNT(DISTINCT host)` + `ORDER BY req_count DESC LIMIT 20`.

6.3 Q3 — Hourly Error Analysis

Per `(log_date, log_hour)`: error count (status $\in [400, 599]$), total count, error rate, and distinct error hosts.

- **MR**: mapper tags `is_error`; reducer computes counts and a hosts-when-error set in one pass.
- **Mongo**: uses `$cond` for the error counter and the `$$REMOVE` sentinel with `$addToSet` so non-error hosts are silently dropped from the set — one-pass like the MR reducer.
- **Pig**: `FILTER tagged BY is_error == 1` inside a nested `FOREACH { DISTINCT }`.
- **Hive**: `COUNT(DISTINCT CASE WHEN status_code BETWEEN 400 AND 599 THEN host END)`.

Optimisation note. `ORDER BY` in Pig and Hive compiles to an additional sampling+sort MR job per query — expensive on a single-node cluster. We removed `ORDER BY` from Q1 and Q3 in Pig (and skipped it similarly in Hive Q1/Q3) because the result sets are small (a few hundred rows); the Python loader sorts before `INSERT`. Q2's `ORDER BY ...LIMIT 20` is retained because it bounds the output.

7. Pipeline Details

7.1 Pipeline 1 — MapReduce (Hadoop Streaming)

Three Python mapper/reducer pairs invoked via the `hadoop-streaming-3.3.6.jar`. Each Q runs as one MR job. The `-D stream.num.map.output.key.fields=2` flag enables two-field composite keys for Q1 and Q3.

7.2 Pipeline 2 — MongoDB Aggregation

The runner stamps each parsed record with `run_id` and `batch_id` and inserts via `insert_many(ordered=False)`. Three compound indexes (`run_id,log_date,status_code` etc.) precede the insert so the aggregations don't COLLSCAN. All three aggregations use `allowDiskUse=True`.

7.3 Pipeline 3 — Apache Pig

Three `.pig` scripts (`q1.pig`, `q2.pig`, `q3.pig`) parameterised by `$input_dir` and `$output_dir`. Each is invoked via `pig -x mapreduce -param ...-f script.pig` sequentially.

7.4 Pipeline 4 — Apache Hive

The bundled `all_queries.hql` script declares one `EXTERNAL TABLE` over the staged HDFS directory and runs three `INSERT OVERWRITE DIRECTORY` statements in a single Hive session — one JVM cold-start serves all three queries. Per-query selection uses split scripts (`q1.hql`, `q2.hql`, `q3.hql`). Hive runs against JDK 8 with a Derby metastore.

8. Major Design Decisions

1. **One CLI, four engines.** A single `main.py` entry point dispatches to engine-specific runners that share the same run-record/batch-log/loader/finalise skeleton. Adding an engine is one new `runner.py` + `loader.py` + scripts dir.
2. **Shared parser at the boundary.** `parse_line` is the single source of truth for log semantics; all four engines see identical records, guaranteeing the project statement's "equivalent semantics" requirement at zero cost.
3. **File-as-batch.** Eliminates the "why 50k?" question by aligning batches with the dataset's natural file boundaries (one month per file).
4. **Separate result tables per query.** Simpler types and queries than a unified table with a discriminator; `pipeline_name` + `execution_time` on each row provides full lineage.
5. **Sort in Python when result is small.** Avoid the per-MR-job sampling+sort cost in Pig/Hive for Q1 and Q3 (bounded outputs).
6. **Bundle Hive queries.** One `hive` JVM cold-start instead of three. The single largest reason Hive beats Pig on wall-clock time on this hardware.
7. **JDK split.** Hive 3.1.3 needs JDK 8; the Hadoop daemons and Python orchestrator use JDK 21. The Hive subprocess gets a `JAVA_HOME=/home/ashok_ubun/jdk8` env var while the rest of the process tree runs on JDK 21.

9. Pipeline Comparison

All four pipelines ran on the same Jul+Aug input (3,457,877 records, 3,736 malformed, 2 batches per run). Q1/Q2/Q3 outputs are byte-identical across all four (verified with set-difference SQL: zero differing rows).

	MapReduce	MongoDB	Pig	Hive
Run ID	1	2	3	4
Runtime	305 s	46 s	1,196 s	212 s
Speedup vs MR	1.0×	6.6×	0.26×	1.4×
Q1 rows	294	294	294	294
Q2 rows	20	20	20	20
Q3 rows	1369	1369	1369	1369
Correctness	baseline	identical	identical	identical

Table 2: Full Jul+Aug 4-way comparison.

9.1 Discussion

Correctness. All four pipelines produce identical Q1, Q2, and Q3 results, byte-for-byte. The shared parser is the key enabler.

Runtime. MongoDB dominates (in-process aggregation, no JVM/YARN tax; plus indexes hit the queries directly). Among Hadoop-based engines, Hive (212 s) beats MR (305 s) because the bundled HQL pays one JVM startup and Hive's planner combines several stages into fewer MR jobs. Pig (1,196 s) is slowest because each of its three `.pig` invocations is a separate JVM cold-start, and the engine's `ORDER BY` compiles into a *sampling+sort* pair of MR jobs — on a single-node WSL cluster, each extra MR submission costs ∼30–60 s of YARN container scheduling that dwarfs the actual compute. Pig's overhead is a single-node artefact; on a larger cluster it would close most of the gap.

Difficulty of implementation. From easiest to hardest:

1. **MongoDB** — aggregation pipelines are dictionary-like; no external job orchestration; debugging via `mongosh`.
2. **Hive** — standard SQL with one `CREATE EXTERNAL TABLE` DDL. The hard part was the Hadoop guava-jar conflict and the JDK 8 vs 21 split.
3. **Pig** — the language is more concise than MR but less familiar; nested `FOREACH` blocks for distinct-within-group take practice. Slower iteration loop because each script is a JVM launch.
4. **MapReduce** — most code (3 mapper + 3 reducer Python files). Two-field composite keys (`stream.num.map.output.key.fields=2`) and explicit `is_error` tagging in the mapper take care.

Suitability for semi-structured log analytics.

- **Hive** is the best general-purpose fit. Declarative SQL handles the entire query set in a single bundled script; schema-on-read with `RegexSerDe` (or, in our case, with the staged TSV) accommodates arbitrary log formats.
- **MongoDB** is the best fit when interactivity matters — ad-hoc `find` or `aggregate` queries return in milliseconds once indexed.
- **Pig** is competitive when transformations are complex and not easily expressed in SQL (nested bags, ad-hoc UDFs).
- **MapReduce** is rarely the right answer in 2026 except when the team needs explicit control over the mapper/reducer execution model; for analytics, Hive/Pig generate the same MR jobs with much less code.

10. How to Run

10.1 Interactive launcher

For demos and casual use, a thin Bash wrapper is provided:

```
./run.sh
```

It opens a menu that walks through pipeline / query / input-file selection, shows a live spinner with elapsed time during the run, and pages the report through `less`. It also has options to start/stop the Hadoop + PostgreSQL + MongoDB services and to browse past run reports. The menu surface is small enough that no separate instruction is needed.

11. Conclusion

The framework cleanly demonstrates the 12 pipeline-×-query combinations required by the problem statement. The shared parser and shared PostgreSQL schema ensure that engine selection is a configuration choice, not a semantics choice — proven by the byte-identical Q1/Q2/Q3 outputs across all four. The file-as-batch interpretation gives a defensible, eval-sanctioned batching unit that needs no arbitrary chunk-size justification. On this single-node WSL cluster, MongoDB and Hive are the fastest; on a real cluster, Hive and Pig would scale further while MongoDB would remain advantaged for interactive queries.